Implementacija RSA Kriptosustava

Dora Tolj

9. siječnja 2025.

Sadržaj

1. Ponavljanje
2. Uvod u RSA
3. Problemi s "običnim metodama"
4. Implementacija algoritama
5. Zaključak
6. Literatura

Osnovna ideja kriptografije javnog ključa

- Kriptografija javnog ključa koristi dva ključa:
  - **Javni ključ**: koristi se za šifriranje poruke.
  - **Privatni ključ**: koristi se za dešifriranje poruke.
- Osigurava sigurnu komunikaciju
- Temelji se na matematičkim problemima (faktorizacija velikih brojeva).
- Konstrukcija kriptosustava kod kojih bi iz poznavanja funkcije šifriranja e_K bilo praktički nemoguće (u nekom razumnom vremenu) izračunati (odrediti) funkciju dešifriranja d_K. Tada bi funkcija e_K mogla biti javna.

Definicija RSA kriptosustava

RSA kriptosustav: Neka je $n = p \cdot q$, gdje su p i q različiti prosti brojevi. Neka

$$P = C = \mathbb{Z}_n;$$
$$\mathcal{K} = \{(n, p, q, d, e) : n = p \cdot q, d \cdot e \equiv 1 \pmod{\varphi(n)}\}.$$

Za $K \in \mathcal{K}$ definiramo:

$$e_K(x) = x^e \mod n,$$
$$d_K(y) = y^d \mod n, \quad x, y \in \mathbb{Z}_n.$$

Vrijednosti n i e su **javne**, a vrijednosti p, q i d su **tajne**, tj. (n, e) je javni, a (p, q, d) je tajni ključ.

Generiranje ključeva

- Odaberu se dva velika prosta broja p i q.
- Računa se $n = p \cdot q$.
- Eulerova funkcija: $\varphi(n) = (p-1)(q-1)$.
- Javni eksponent e odabire se tako da je relativno prost s $\varphi(n)$.
- Privatni eksponent d je modularni inverz od e prema $\varphi(n)$.

Matematičke operacije

- **Šifriranje:** $c = m^e \mod n$
- **Dešifriranje:** $m = c^d \mod n$
- Gdje su:
  - m - poruka (broj manji od n)
  - c - šifrirana poruka

Zašto RSA?

- Osigurava tajnost komunikacije.
- Ne zahtijeva dijeljenje privatnog ključa.
- Temeljen na problemu faktorizacije velikih brojeva.
- U samom se šifriranju i dešifriranju koristi modularno potenciranje, dok se faktorizacija koristi u dobivanju dodatnog podatka (“trapdoora“)

Zašto ne možemo koristiti običnu metodu množenja pa modularnog operiranja?

- Kada radimo s velikim brojevima (poput 2048-bitnih brojeva u RSA), obična metoda množenja može brzo postati nepraktična.
- Problem je u tome što se prilikom običnog množenja rezultati potenciranja brzo povećavaju u veličini, što može uzrokovati povećanje vremena obrade i memorijske zahtjeve.
- Kako?

Vrijeme i memorijski zahtjevi s običnom metodom množenja pa moduliranja

- Pogledajmo prvo primjer običnog množenja i moduliranja na manjim brojevima.
- Pretpostavimo da imamo a=5 (baza), b=20 (eksponent), n=13.
- Broj 5^{20}= 95367431640625 ima 15 znamenki
- Već smo spomenuli da RSA koristi vrlo velike brojeve, često 2048-bitne.
- Što znači veličina od 2048 bita?

Problem veličine rezultata

- Veličina od 2048 bita znači da je n broj koji se može zapisati s 2048 binarnih znamenki.
- $2^{2048} \approx 3.23 \times 10^{616}$

PROBLEM VELIČINE REZULTATA

- Ako je a veličine 2048 bita i b također 2048 bita, tada a^b ima otprilike $b \cdot \log_2 a$ bitova.
- U najgorem slučaju, a^b ima oko $2048 \times 2048 = 4,194,304$ bita.

Problemi kod računala

- Množenje dvaju k-bitnih brojeva ima vremensku složenost $O(k^2)$ za standardne algoritme.
- Ako je $k = 4,194,304$, to znači $O(k^2) = 4,194,304^2$ je približno 17.6×10^{12} operacija.
- Moderni CPU može obaviti milijardu operacija u sekundi (10^9).
- Jedno množenje (17.6×10^{12}) podijeljeno s 10^9 daje $17,600$ sekundi, odnosno oko 5 sati.
- S obzirom na to da eksponent b ima 2048 bita, ukupno vrijeme postaje: 2048×5 sati, što je približno $10,240$ sati ili 426 dana!

Primjer - HTTPS

- Web stranice koje koriste HTTPS koriste SSL/TLS
- RSA se koristi za sigurnu razmjenu ključeva
- Javni ključ i privatni ključ obično veličine 2048 bitova

Problemi na web stranicama

- Visoka potrošnja procesorskog vremena
- Opasnost od preopterećenja
- Važnost optimiziranih algoritama!

Square-and-Multiply Algoritam

- Algoritam za efikasno modularno potenciranje.
- Koristi se za šifriranje i dešifriranje u RSA
- Neka želimo izračunati $x^k \mod n$
- Osnovni koraci:
  - Inicijaliziraj $r = 1$ (početna vrijednost rezultata).
  - Za svaki bit k_i eksponenta k (od najznačajnijeg prema najmanje značajnom):
    - Kvadriraj trenutnu vrijednost r
    - Ako je bit $k_i = 1$, pomnoži s bazom x
  - Na kraju, r predstavlja rezultat $x^k \mod n$.

Pseudokod: Square-and-Multiply

Ulaz: x, k, n *(baza, eksponent, modul)*
Izlaz: $r = x^k \mod n$

1. $r \leftarrow 1$ *(Inicijalizacija rezultata)*
2. Za svaki *bit* u binarnom prikazu *eksp* (od najvažnijeg prema najmanje važnom):
   1. $r \leftarrow (r \cdot r) \mod n$ *(Kvadriraj trenutni rezultat)*
   2. Ako je $bit = 1$, tada:
      1. $r \leftarrow (r \cdot x) \mod n$ *(Množi s bazom)*
3. Vrati r *(Konačni rezultat)*

Prednosti i mane algoritma

- **Prednosti:**
  - Efikasan za velike eksponente.
  - Smanjuje broj potrebnih operacija u odnosu na obični algoritam za potenciranje.
  - Osigurava konstantno vrijeme za svaku operaciju
- **Mane:**
  - Potrebno je puno računanja za velike eksponente s velikim brojem bitova.
  - Manje efikasan od naprednijih algoritama poput Montgomery Laddera u nekim slučajevima.

Montgomery Ladder

- Algoritam za efikasno modularno potenciranje
- Bolji je od Square-and-Multiply jer Montgomery Ladder koristi isti set operacija za svaki bit eksponenta
- Upotrebljava dva registra i radi u dvije faze - optimizira broj operacija za računanje

Osnovna ideja Montgomery Ladder algoritma

- Pretpostavimo da želimo izračunati $\mathbf{x^e} \mod \mathbf{n}$, gdje je e eksponent.
- Koristimo **dva** registra: jedan za privremeni rezultat R_0 i drugi za trenutnu bazu x R_1.
- Algoritam čita eksponent bit po bit i, ovisno o vrijednosti bita (0 ili 1), obavlja određene operacije:
  - **Ako je bit = 1**, množi trenutni rezultat R_0 s trenutnom bazom x $(mod n)$, a registar R_1 kvadriraj.
  - **Ako je bit = 0**, samo kvadrira R_0 $(mod n)$,a R_1 množi s R_0.
- Na kraju, vraćamo rezultat u registar R_0, koji predstavlja $x^e \mod n$.

Pseudokod

Funkcija MontgomeryLadder(x, e, n)

1. $R_0 \leftarrow 1$
2. $R_1 \leftarrow x$
3. **Za svaki bit** i broja e (od najvišeg prema najnižem):
   1. **Ako** je i-ti bit jednak 1:
      - $R_0 \leftarrow (R_0 \cdot R_1) \bmod n$
      - $R_1 \leftarrow (R_1 \cdot R_1) \bmod n$
   2. **Inače**:
      - $R_0 \leftarrow (R_0 \cdot R_0) \bmod n$
      - $R_1 \leftarrow (R_0 \cdot R_1) \bmod n$
4. **Vrati** R_0

Multiplicative Splitting Modulo N

- Želimo izračunati $x^e \mod N$
- Prvo ćemo objasniti pomoćne metode multiplicative splitting i modular exponentiation koje će nam koristiti u algoritmu modular exponentiation with splitting(sve metode korištene su u kodu)
- Multiplicative splitting je metoda kojom razdvajamo broj x na dva manja broja x_0 i x_1, takva da:

$$x = x_0^{-1} \times x_1 \mod N$$

- Ovi brojevi zadovoljavaju uvjet $|x_0|, |x_1| \leq \lceil \sqrt{N} \rceil$.
- Algoritam koristi prošireni Euklidov algoritam kako bi pronašao vrijednosti x_0 i x_1.

- Inicijaliziraju se početne vrijednosti:
  - $r_0 = N$, $r_1 = x$ (ostatci),
  - $a_0 = 0$, $a_1 = 1$ (koeficijenti vezani uz x).
- U svakoj iteraciji izračunavaju se:
  - Novi ostatak: $r_{i+1} = r_{i-1} - q_i \cdot r_i$,
  - gdje je $q_i = \lfloor r_{i-1}/r_i \rfloor$ (cjelobrojni kvocijent),
  - Novi koeficijenti: $a_{i+1} = a_{i-1} - q_i \cdot a_i$.
- Ovdje zaustavljamo algoritam kada su r_i i a_i manji od $\sqrt{N}$.
- Na kraju, dobivamo brojeve
  - $x_0 = a_1$ (prvi broj razdvajanja),
  - $x_1 = r_1$ (drugi broj razdvajanja).

Modular exponentiation

1. **Cilj algoritma:**
   - Računanje $x^e \mod n$
2. **Princip rada:**
   - Eksponent se razlaže na manje dijelove.
   - Baza se kvadrira i eksponent se dijeli s 2.
   - Ako je eksponent neparan, trenutni rezultat se množi s bazom.
3. **Koraci algoritma:**
   - Inicijalizacija: $\text{rezultat} = 1$, baza se reducira modulo n.
   - Iteracija:
     - Ako je $e \mod 2 = 1$: $\text{rezultat} = (\text{rezultat} \cdot \text{baza}) \mod n$,
     - Kvadrira se baza: $\text{baza} = (\text{baza} \cdot \text{baza}) \mod n$,
     - Eksponent se dijeli s 2: $e// = 2$.
   - Kada je $e = 0$, vraća se rezultat.

Modularno potenciranje s razdvajanjem

1. **Cilj algoritma:**
   - Računanje $x^e \mod N$ uz primjenu razdvajanja x na x_0 i x_1.
2. **Koraci algoritma:**
   - **Razdvajanje broja x:** $x = x_0^{-1} \cdot x_1 \mod N$.
   - **Inverz x_0:** Računa se $x_0^{-1} \mod N$.
   - **Modularno potenciranje:**
     - prvi dio $= (x_0^{-1})^e \mod N$,
     - drugi dio $= x_1^e \mod N$.
   - **Spajanje rezultata:**

$$\text{rezultat} = (\text{prvi dio} \cdot \text{drugi dio}) \mod N$$

Pseudokod

Funkcija: modular exponentiation with splitting
Ulaz: x, e, N
Izlaz: $x^e \mod N$

- $x_0, x_1 \leftarrow$ multiplicative splitting(x, N)
- $x_0^{-1} \leftarrow$ modularna inverzija(x_0, N)
- *prvi dio* $\leftarrow$ modular exponentiation(x_0^{-1}, e, N)
- *drugi dio* $\leftarrow$ modular exponentiation(x_1, e, N)
- *rezultat* $\leftarrow$ (prvi dio $\times$ drugi dio) $\mod N$
- **Vrati:** rezultat

Zaključak

- RSA je temelj modernih sigurnosnih protokola.
- Implementacija zahtijeva upotrebu optimiziranih algoritama.
- Algoritmi poput Square-and-Multiply, Montgomery Ladder i Multiplicative Splitting modulo N unaprjeđuju sigurnost i performanse.
- Budućnost RSA-a ovisi o otpornosti na kvantne napade.

Literatura

- https://web.math.pmf.unizg.hr/~duje/kript/rsa.html
- https://hal.archives-ouvertes.fr/hal-01185249/document
- https://www.youtube.com/watch?v=cbGB__V8MNk&t=908s
- https://chatgpt.com/